Multilingual RAG for Government Schemes

Name	Roll No.	Contribution
Aakashdeep Paul	2330055	Data collection: English and Hindi scheme documents, metadata and source verification; labelled evaluation questions
Ananwita Dash	2330067	Ingestion and section-aware chunking, multilingual embeddings, BM25 and hybrid retriever
Palak Sengar	2330097	Cited answer generation (extractive and LLM), language and intent detection, eligibility checker
Sneha Singh	2330120	Streamlit web application, evaluation script, project report, slides and demonstration video

Problem Statement and Objectives

Problem: The Government of India runs hundreds of welfare schemes, but the rules for who can benefit are spread across long portal pages and guideline PDFs. Most of this material is written in English, while a large share of citizens are more comfortable asking in Hindi. A citizen who simply wants to know *"Am I eligible, and what do I get?"* has to read several documents, and generic chatbots may answer confidently without any source.

Motivation: Retrieval-Augmented Generation (RAG) grounds every answer in retrieved passages, so each claim can be traced back to an official document. Multilingual sentence embeddings place Hindi and English text in one vector space, so a question in one language can find the answer in a document written in the other.

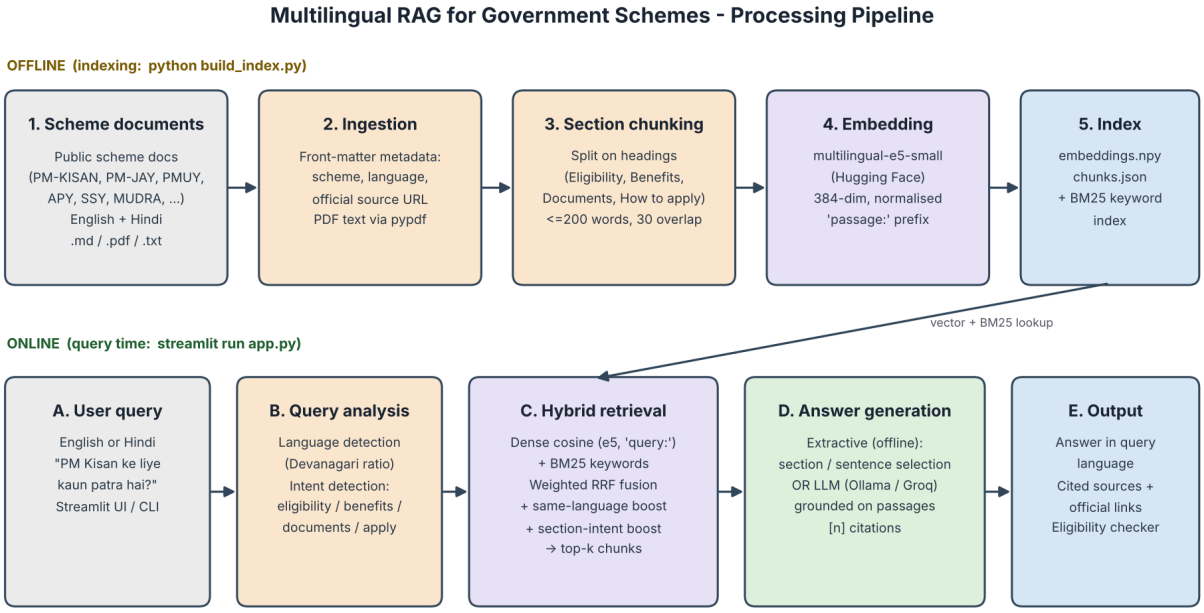
Objectives:

1. Build a corpus of publicly available central government scheme documents in English and Hindi.
2. Index the documents with multilingual embeddings so that both English and Hindi queries are supported, including cross-lingual retrieval.
3. Retrieve the most relevant passages with a hybrid (dense + keyword) retriever that favours the section the user is asking about, such as Eligibility.
4. Generate answers with a Large Language Model (LLM) that uses only the retrieved passages, in the user's language, with numbered citations to the scheme, section and official source URL.
5. Provide a usable web interface, including a profile-based eligibility checker, and evaluate retrieval quality on labelled English and Hindi questions.

Expected outcomes: A working prototype that runs locally in VS Code, answers scheme questions in English and Hindi with cited eligibility information, works offline after setup, and reports measurable retrieval accuracy.

Proposed Methodology and Processing Pipeline

The system has two phases: an offline phase that turns scheme documents into a searchable index, and an online phase that answers each question from that index with citations.



Offline stages:

1. **Scheme documents-** Ten central schemes are written as structured Markdown in English and Hindi (20 files), summarised from their official portals. Official PDFs can also be added; their text is extracted with pypdf.
2. **Ingestion-** Each file's front matter gives the scheme id, title, language, ministry and official source URL. These travel with every chunk so answers can be cited.
3. **Section-aware chunking-** Documents are split on their headings (Overview, Benefits, Eligibility, Exclusions, Documents Required, How to Apply). Long sections are cut into windows of at most 200 words with a 30-word overlap, giving 112 chunks. Each chunk keeps its section name.
4. **Multilingual embedding-** Each chunk, prefixed with its title and section, is encoded by multilingual-e5-small into a 384-dimensional normalised vector.
5. **Index-** Vectors, chunk texts and metadata are saved to disk, and a BM25 keyword index is built over the same chunks.

Online stages:

1. **User questions-** in English or Hindi.
2. **Query analysis-** The language is detected from the share of Devanagari characters. Keyword rules detect the intent: eligibility, benefits, documents or how to apply. Eligibility takes priority, because eligibility questions often also contain benefit words.
3. **Hybrid retrieval-** The question is embedded and compared with every chunk by cosine similarity, and BM25 scores the same chunks by keyword overlap. The two rankings are merged with weighted Reciprocal Rank Fusion (dense 0.6, BM25 0.4, k = 60). Chunks in the question's language and chunks from the section matching the intent each get a +0.15 boost. The top-k chunks (default 5) are returned.

4. **Answer generation-** The top-k passages are numbered and sent to an LLM (Llama-3.3-70B via the Groq API by default; Google Gemini or a local Ollama model can also be used). The prompt instructs the LLM to answer only from these passages, in the question's language, to list who is eligible and who is excluded for eligibility questions, to add a [n] citation after every fact, and to say so if the answer is not in the passages. Temperature is 0.1 to keep answers factual. If no LLM is connected or it is unreachable, an offline extractive generator is used instead: it returns the matching section verbatim, or the most relevant sentences, each with a [n] citation.
5. **Output-** The answer in the user's language, the numbered sources with section, retrieval scores and official link, and a disclaimer to verify on the official portal.

Input, intermediate data and output

Stage	Data
Input	A natural-language question in English or Hindi, for example "पीएम किसान योजना के लिए कौन पात्र है?"
Intermediate	Detected language and intent; query embedding (384-dim); dense and BM25 scores for 112 chunks; fused ranking; top-5 chunks with metadata; prompt with the numbered passages sent to the LLM
Output	Cited answer in the question's language; source list with scheme, section, language, scores and official URL

4. Models and Tools Used

The system uses two pre-trained models: a multilingual sentence-embedding model from Hugging Face for retrieval, and an LLM for answer generation. No model is fine-tuned, so the system runs on a CPU laptop.

Models

Component	Model / method	Details	Source
Multilingual embeddings	intfloat/multilingual-e5-small	118M parameters, 384-dim vectors, about 100 languages including Hindi; uses "query:" and "passage:" prefixes	Hugging Face
Keyword retrieval	Okapi BM25 (k1 = 1.5, b = 0.75)	Own NumPy implementation with an English + Devanagari tokeniser and stop-word lists	Robertson and Zaragoza, 2009
Rank fusion	Weighted Reciprocal Rank Fusion	Dense weight 0.6, BM25 weight 0.4, k = 60, plus language and section boosts	Cormack et al., 2009
Answer generation (offline fallback)	Extractive generator	Returns the matching section or the best sentences, scored by embedding similarity + keyword overlap	This project

Component	Model / method	Details	Source
Answer generation (LLM)	Llama-3.3-70B-Versatile via Groq; Gemini 2.5 Flash; Qwen2.5-3B via Ollama	OpenAI-compatible chat API; temperature 0.1; answers only from the retrieved passages, in the user's language, with [n] citations; model chosen in the app sidebar	Groq, Google AI Studio, Ollama
Alternative embeddings	paraphrase-multilingual-MiniLM-L12-v2, BAAI/bge-m3	Selectable with the EMBEDDING_MODEL setting	MiniLM, bge-m3

Libraries and frameworks

Library	Used for
sentence-transformers, PyTorch, Hugging Face transformers	Loading and running the embedding model
NumPy	Vector search, BM25 scoring, rank fusion
scikit-learn	Character n-gram TF-IDF fallback when the embedding model is unavailable
pypdf	Text extraction from official scheme PDFs
Streamlit	Web interface
requests	Calls to the LLM API (Groq / Gemini / Ollama)
Matplotlib	Evaluation chart and pipeline diagram

Implementation environment- Python 3.11 on Windows 11, developed and run in Visual Studio Code with a project virtual environment (.venv). Runs on CPU; about 1.5 GB of disk is needed for PyTorch and the model. After the first run, which downloads the model (about 470 MB), the system works offline. The LLM generation step needs an internet connection and a free API key (Groq or Gemini), or a locally installed Ollama model.

Implementation and Usability

The system is a modular Python package (src/) with four entry scripts and a Streamlit web app; it runs locally in VS Code with one-click launch configurations.

Implementation procedure:

1. Collected scheme information from official portals and wrote one structured Markdown file per scheme and language, with front matter for scheme id, title, language, ministry and source URL.
2. Implemented loading and section-aware chunking (src/ingest.py), and language detection, tokenisation and intent rules for English and Hindi (src/text_utils.py).
3. Wrapped the multilingual embedding model with query and passage prefixes and a TF-IDF fallback (src/embeddings.py), implemented BM25 (src/bm25.py), and saved the index to disk (src/index.py).
4. Built the hybrid retriever with weighted rank fusion and boosts (src/retriever.py).
5. Built the extractive and LLM answer generators with citations and Hindi/English disclaimers (src/generator.py), joined by the end-to-end pipeline (src/pipeline.py).
6. Added a rule-based eligibility screener (src/eligibility.py), the Streamlit interface (app.py), a terminal interface (cli.py), and the evaluation script (evaluate.py).

Code structure:

File	Role
build_index.py	Reads all documents, chunks, embeds and saves the index
app.py	Streamlit web app: Ask, Eligibility checker, How it works
cli.py	Terminal question answering
evaluate.py	Metrics on 44 labelled questions, results table and chart
make_samples.py	Generates sample inputs and outputs
src/	Configuration, ingestion, embeddings, BM25, retriever, generator, eligibility, pipeline
data/schemes/en, data/schemes/hi	20 scheme documents
eval/questions.json	22 English + 22 Hindi labelled questions

Instructions for running the application:

1. Open the project folder in VS Code and create a virtual environment: python -m venv .venv, then .\.venv\Scripts\Activate.ps1 (Windows).
2. Install dependencies: pip install -r requirements.txt.
3. Build the index: python build_index.py. The first run downloads the embedding model.
4. Start the app: streamlit run app.py, which opens at http://localhost:8501.
5. Optional: python cli.py for terminal Q&A, python evaluate.py for metrics, python make_samples.py for sample outputs. All are also available from VS Code's Run and Debug panel.

6. Connect the LLM: get a free API key from console.groq.com/keys, open "LLM (generation step)" in the app sidebar, choose Groq, paste the key and click Connect. The key is saved in a .env file that is not uploaded to GitHub.

Using the app:

- **Ask a question:** type a question in English or Hindi, or click an example. The answer shows numbered citations; the Sources panel lists scheme, section, language, retrieval scores and the official link.
- **Settings sidebar:** number of passages (top-k), answer language, which document language to search (used to demonstrate cross-lingual retrieval), scheme filter, and uploading new PDFs to re-index.
- **LLM (generation step):** choose the provider (Groq, Google Gemini or Ollama), paste the API key and connect. The "Use LLM for answer" switch turns LLM generation on or off, and the caption under each answer shows which generator was used.
- **Eligibility checker:** enter age, gender, area, occupation and household details. The app lists schemes as likely eligible, to be checked, or not eligible, each with the reason and the cited eligibility text.

Practical applications:

- Self-service help for citizens who prefer to ask in Hindi.
- A support tool for Common Service Centre operators, gram panchayat staff and NGO field workers who explain schemes to beneficiaries.
- A grounded, citable alternative to general chatbots: every claim links to a section and official URL.
- Easily extended: dropping new official PDFs into the data folder adds schemes without code changes.

Results and Sample Outputs

The system answers English and Hindi questions from the correct scheme section with citations, and supports Hindi questions over English documents. Representative outputs follow (top-k = 5, extractive mode, multilingual-e5-small).

Sample 1 — English eligibility question

Input: Who is eligible for PM-KISAN?

Output: Pradhan Mantri Kisan Samman Nidhi (PM-KISAN). **Eligibility:** All landholding farmer families that have cultivable land in their names are eligible; a family is husband, wife and minor children [1]. **Exclusions:** institutional landholders; holders of constitutional posts; ministers, MPs, MLAs, mayors and district panchayat chairpersons; serving or retired government employees (except MTS, Class IV and Group D); pensioners receiving Rs 10,000 or more per month; income-tax payers; and registered professionals such as doctors, engineers, lawyers, CAs and architects [2].

Sources: [1] PM-KISAN — Eligibility (en), [2] PM-KISAN — Exclusions (en), pmkisan.gov.in.

Sample 2 — Hindi eligibility question

Input: पीएम किसान योजना के लिए कौन पात्र है?

Output: प्रधानमंत्री किसान सम्मान निधि (पीएम-किसान)। पात्रता: वे सभी भूमिधारक किसान परिवार पात्र हैं जिनके नाम पर खेती योग्य भूमि है [1]। अपात्रता: संस्थागत भूमिधारक, संवैधानिक पदधारक, सरकारी कर्मचारी, 10,000 रुपये या अधिक पेंशन वाले, आयकरदाता और पंजीकृत पेशेवर [3]।

Sources: [1] पीएम-किसान — पात्रता (hi), [3] पीएम-किसान — अपात्रता (hi). The language was detected as Hindi and the answer came from the Hindi document.

Sample 3 — Eligibility question with an exclusion

Input: Can an income tax payer join Atal Pension Yojana?

Output: Atal Pension Yojana (APY). **Eligibility:** any citizen of India aged 18 to 40 with a savings bank or post office account; from 1 October 2022, any citizen who is or has been an income tax payer is not eligible to join APY [1].

Sample 4 — Hindi benefits question

Input: मनरेगा में साल में कितने दिन काम की गारंटी है?

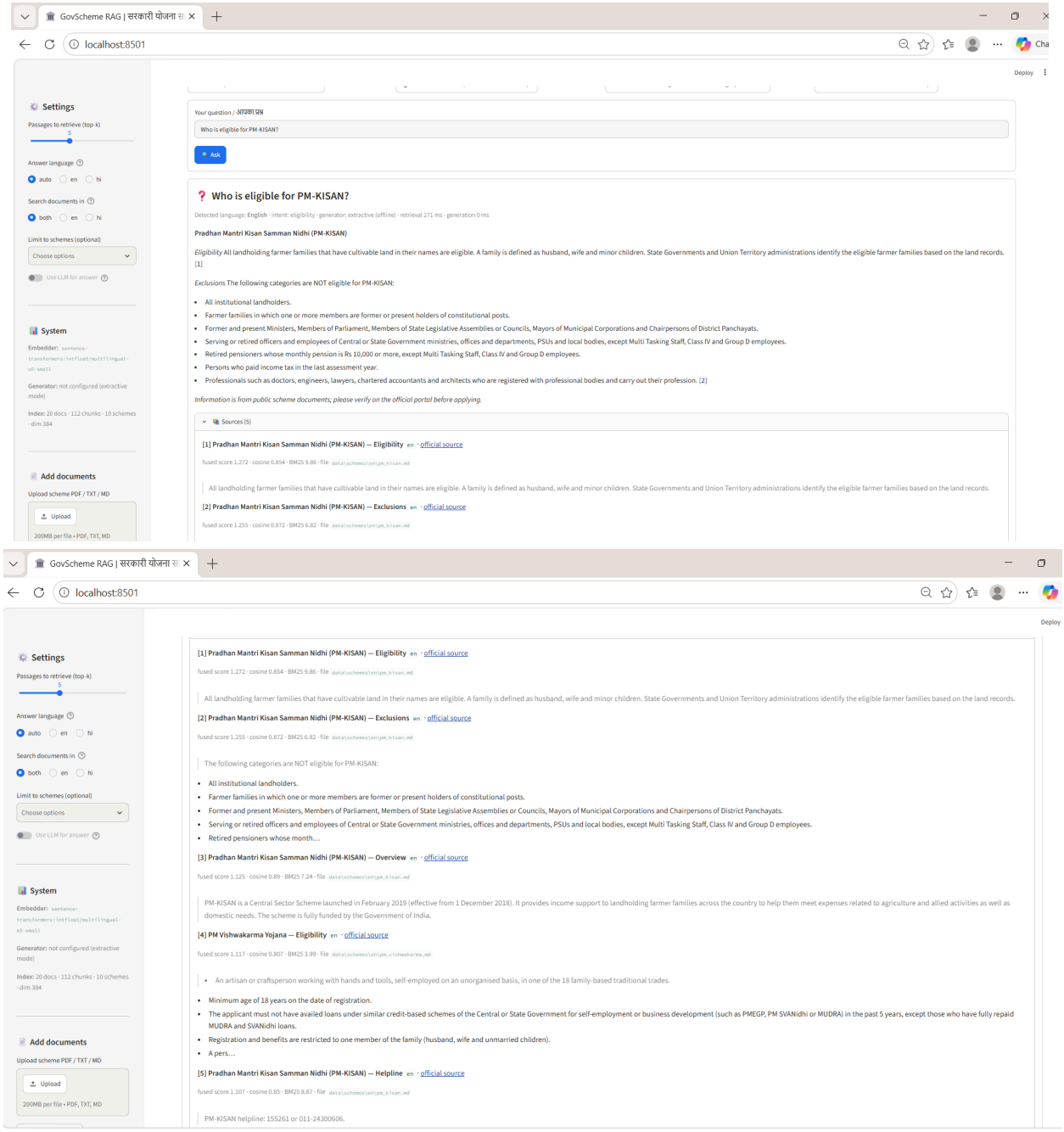
Output: मनरेगा, लाभ: हर ग्रामीण परिवार को एक वित्त वर्ष में कम से कम 100 दिन का गारंटीड मजदूरी रोजगार; 15 दिन में काम न मिलने पर बेरोजगारी भत्ता; 5 किलोमीटर से दूर काम पर 10 प्रतिशत अतिरिक्त मजदूरी [1]।

Sample 5 — Cross-lingual retrieval

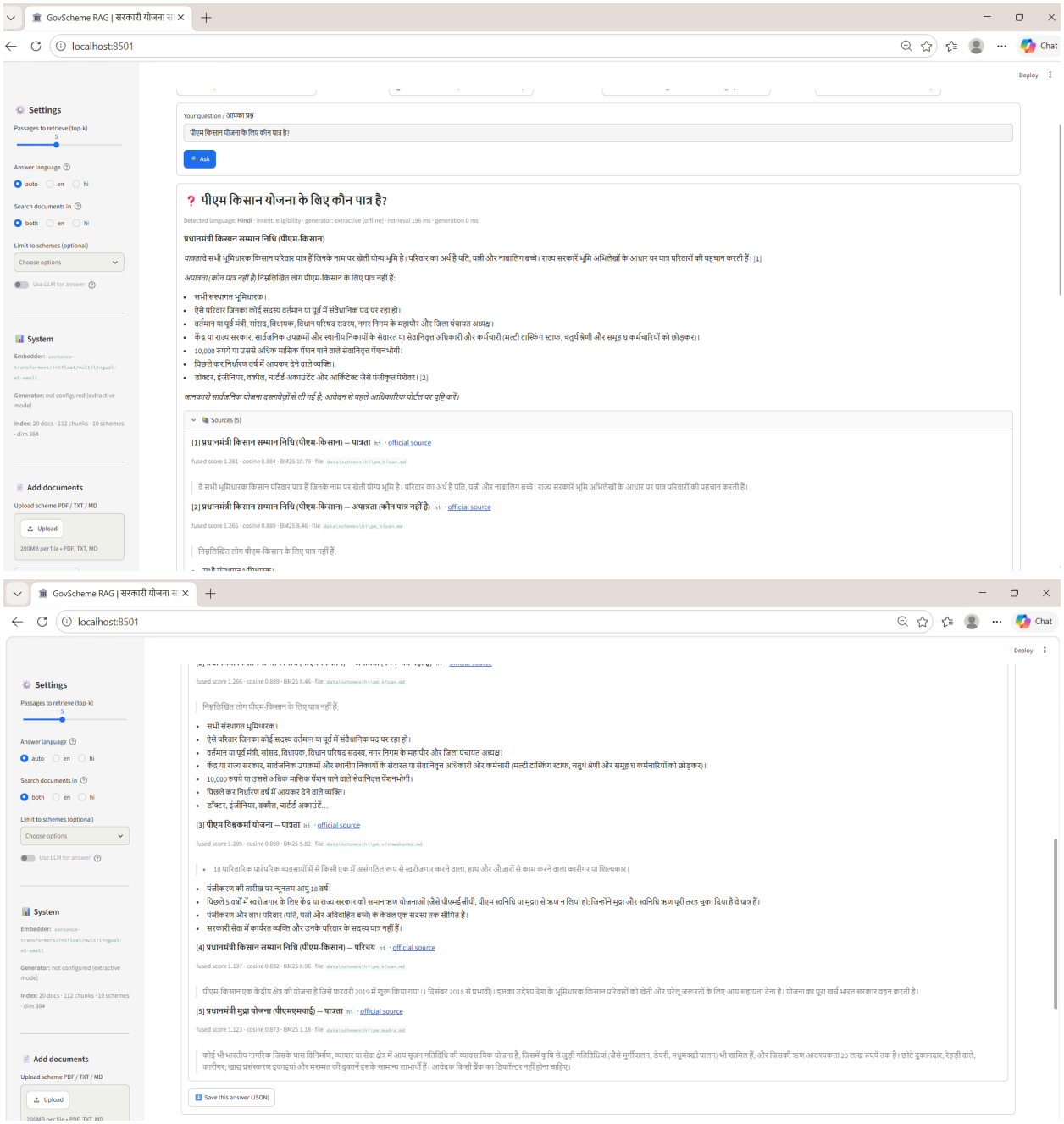
Input: पीएम किसान योजना के लिए कौन पात्र है? with "Search documents in" set to English only.

Output: The PM-KISAN English Eligibility and Exclusions sections are retrieved, with a note that the matching passages were found in English. This works only because the multilingual embeddings map the Hindi question close to the English passage.

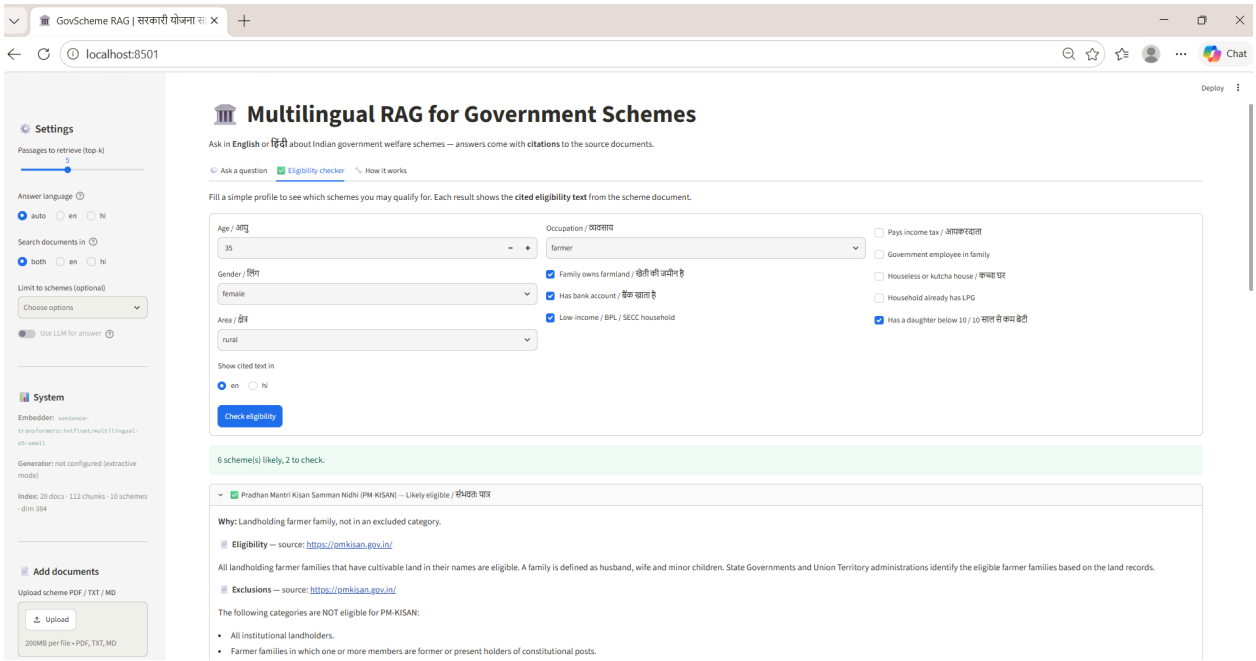
Screenshots



Sample 1 and its Sources panel



Hindi answer



6 scheme(s) likely, 2 to check.

Pradhan Mantri Kisan Samman Nidhi (PM-KISAN) — Likely eligible / संभवतः पात्र

Why: Landholding farmer family, not in an excluded category.

Eligibility — source: https://pmkisan.gov.in/

All landholding farmer families that have cultivable land in their names are eligible. A family is defined as husband, wife and minor children. State Governments and Union Territory administrations identify the eligible farmer families based on the land records.

Exclusions — source: https://pmkisan.gov.in/

The following categories are NOT eligible for PM-KISAN:

- All institutional landholders.
- Farmer families in which one or more members are former or present holders of constitutional posts.
- Former and present Ministers, Members of Parliament, Members of State Legislative Assemblies or Councils, Mayors of Municipal Corporations and Chairpersons of District Panchayats.
- Serving or retired officers and employees of Central or State Government ministries, offices and departments, PSUs and local bodies, except Multi Tasking Staff, Class IV and Group D employees.
- Retired pensioners whose monthly pension is Rs 10,000 or more, except Multi Tasking Staff, Class IV and Group D employees.
- Persons who paid income tax in the last assessment year.
- Professionals such as doctors, engineers, lawyers, chartered accountants and architects who are registered with professional bodies and carry out their profession.

Pradhan Mantri Ujjwala Yojana (PMUY) — Likely eligible / संभवतः पात्र

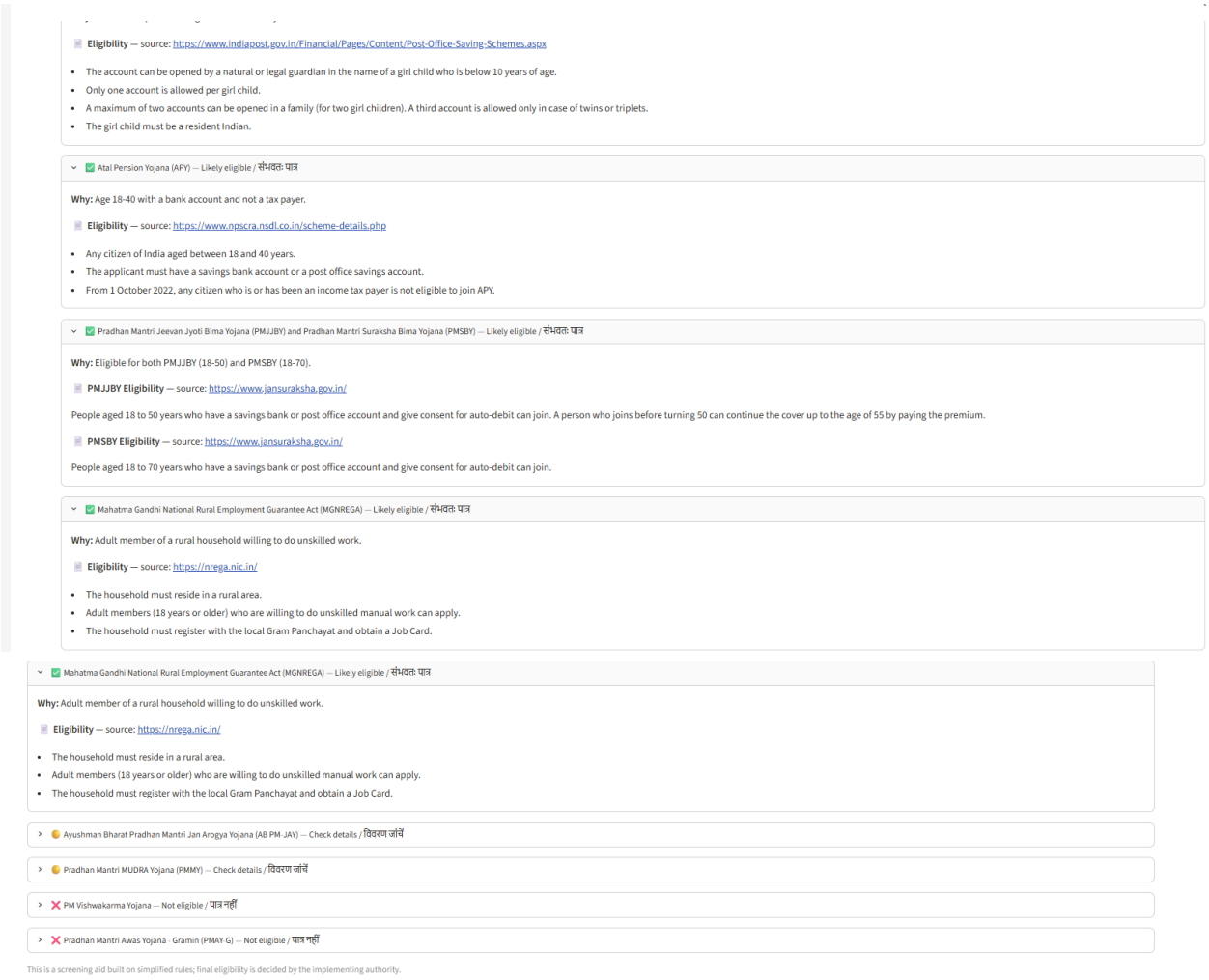
Why: Adult woman from a poor household without LPG.

Eligibility — source: https://www.pmuy.gov.in/

The applicant must be a woman who has attained 18 years of age and belongs to a poor household. There must be no other LPG connection in the same household. She must belong to one of the following categories:

- Scheduled Castes or Scheduled Tribes households.
- Beneficiaries of Pradhan Mantri Awas Yojana (Gramin).
- Antyodaya Anna Yojana (AAY) households.
- Most Backward Classes, forest dwellers, residents of islands and river islands, and tea and ex-tea garden tribes.
- Households listed in SECC 2011.
- Any other poor household that submits a 14-point declaration of poverty.

Sukanya Samriddhi Yojana (SSY) — Likely eligible / संभवतः पात्र



Eligibility checker results

Performance evaluation

Retrieval was evaluated on 44 labelled questions (22 English, 22 Hindi) covering all 10 schemes and four intents. Metrics: Hit@1 and Hit@3 (correct scheme at rank 1 or within the top 3), MRR (mean reciprocal rank), Section@1 (the top passage is from the asked-about section) and FactRecall (the expected fact, such as "Rs 6,000", appears in the answer). Four configurations are compared: hybrid (the full system), dense only, BM25 only, and cross-lingual (each question searched only in documents of the other language).

Configuration	Hit@1	Hit@3	MRR	Section@1	FactRecall
Hybrid (all)	0.955	1.0	0.977	0.864	0.909
Hybrid (English)	1.0	1.0	1.0	0.955	0.955
Hybrid (Hindi)	0.909	1.0	0.955	0.733	0.864
Dense only	0.955	0.977	0.97	0.841	—

Configuration	Hit@1	Hit@3	MRR	Section@1	FactRecall
BM25 only	0.864	1.0	0.924	0.75	—
Cross-lingual	0.864	0.955	0.919	0.727	—

Observations:

- Section-aware chunking lets eligibility questions return the complete Eligibility and Exclusions text instead of fragments, which is what makes the answers citable.
- Intent detection matters: questions such as "Can an income tax payer join APY?" contain benefit words ("pension"), so eligibility is given priority over benefits.
- BM25 helps with exact scheme names and numbers (PMJJBY, Rs 436), while dense embeddings handle paraphrases and Hindi-English mismatches; the hybrid combines both.
- Retrieval takes about 10 ms per question on a laptop CPU once the model is loaded.

Limitations:

- The corpus covers 10 central schemes as curated summaries of official portals; amounts and rules change, so every answer carries a "verify on the official portal" note.
- LLM generation needs internet access and an API key (or a local Ollama model); without it, the app falls back to extractive answers, which return whole sections rather than a personalised yes/no. An LLM can occasionally phrase things loosely, so the cited source passages are always shown beside the answer.
- The eligibility checker uses simplified rules and is a screening aid, not an official decision.
- Hindi written in Roman script (Hinglish) is handled only partially.
- Scanned, image-only PDFs need OCR before they can be indexed.
- The first setup downloads about 1 GB (PyTorch and the model), which is slow on weak connections.

Conclusion and Future Scope

We built a working multilingual RAG prototype that answers English and Hindi questions about 10 Indian government schemes with cited eligibility information, running offline on a laptop CPU.

Summary of achievements:

- A bilingual corpus of 20 structured scheme documents with official source links.
- Cross-lingual retrieval through multilingual-e5-small embeddings, combined with BM25 in a hybrid retriever with language and section-intent boosts.
- LLM-generated answers grounded only on the retrieved passages, in the user's language, with numbered citations to the scheme, section and official URL, plus an offline extractive fallback.
- A Streamlit app with Q&A, a profile-based eligibility checker and document upload, plus a terminal interface and an evaluation suite of 44 labelled questions.

Possible improvements and extensions:

- More Indian languages such as Odia, Bengali, Tamil and Telugu; the embedding model already supports them, so only documents are needed.
- State government schemes, and automatic updates from myScheme and official portals.
- A cross-encoder re-ranker and a larger embedding model (bge-m3) for higher accuracy.
- Voice input and spoken answers (Whisper speech recognition and text-to-speech) for users with low literacy.
- OCR for scanned PDFs, and better handling of Hinglish (Hindi typed in Roman script).
- Deployment on Hugging Face Spaces or as a WhatsApp chatbot for wider access.

References and GitHub Repository Link

GitHub repository: https://github.com/Ananwita/Govscheme-rag

Papers and methods:

1. L. Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," arXiv:2402.05672, 2024. arxiv.org/abs/2402.05672
2. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020. arxiv.org/abs/2005.11401
3. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, 2009.
4. G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods," SIGIR, 2009.
5. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP, 2019. arxiv.org/abs/1908.10084

Models and libraries:

6. intfloat/multilingual-e5-small — huggingface.co/intfloat/multilingual-e5-small
7. sentence-transformers — sbert.net
8. Streamlit — streamlit.io
9. LLM APIs: Groq (Llama-3.3-70B) — console.groq.com; Google Gemini — aistudio.google.com; Ollama (local runtime) — ollama.com

Data sources (official scheme portals):

10. PM-KISAN — pmkisan.gov.in
11. Ayushman Bharat PM-JAY — nha.gov.in/PM-JAY
12. Pradhan Mantri Ujjwala Yojana — pmuy.gov.in
13. Sukanya Samriddhi Yojana — indiapost.gov.in
14. Atal Pension Yojana — npscra.nsdl.co.in
15. PMJJBY and PMSBY — jansuraksha.gov.in
16. PM MUDRA Yojana — mudra.org.in
17. MGNREGA — nrega.nic.in
18. PM Vishwakarma — pmvishwakarma.gov.in
19. PMAY-Gramin — pmayg.nic.in
20. myScheme national scheme platform — myscheme.gov.in